DR03: The Puzzler

DEADLINES

Release	*Mar 17, 2026*
Crit	*None*
Due Date	Mar 30, 2026

TEACHING OBJECTIVES

By completing this assignment, you will be able to:

- Use spread (...), object literals ({}), and array methods (e.g. .map(), .filter()) to build and transform data.
- **Create and reuse React components** that receive props and return JSX.
- **Move data between component** by passing props from parent to child.
- **Update state** with useState so the UI reacts to user actions.
- Implement **drag and drop** using the browser's Drag and Drop API.
- **Snap** draggable pieces to a grid so the puzzle has a clear layout.

OVERVIEW

In this assignment, you will build a **fully functional jigsaw-style puzzle UI** across six problems. Each problem focuses on one React or JavaScript concept; together they lead to a working puzzle you can drag, drop, and snap to a grid.

You will:

- Problems 1–3: Build puzzle data with spread and array methods, create a `PuzzlePiece` component, and pass piece data from parent to child components.

- Problem 4: Use state to hold piece order and react to a "Shuffle" (or similar) action.

- Problem 5: Add drag and drop so pieces can be moved between slots.

- Problem 6: Implement snap to grid so dropped pieces align to fixed slots.

The problems are in src/problems/problem-1.jsx through problem-6.jsx. Use the dropdown in the app to switch between them. Complete each in order; later problems build on the same puzzle structure.

WARNING

DO NOT USE AI to blanket solve the problems. You need to be able to inspect, understand, and reason through React code for your Final Exam.

LOGISTICA

- Clone https://github.com/cearto/puzzler.
- Run a codespace.

PROBLEM 1 : HELLO PUZZLE LIST

Expected behavior:

Problem 1: ..., {}, and array methods

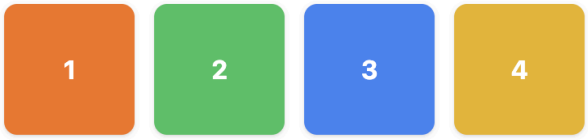
Build puzzle piece data with object literals and `.map()`. Then render a list of piece labels.

- Piece 1
- Piece 2
- Piece 3
- Piece 4

PROBLEM 2 : PuzzlePiece

Problem 2: Creating components

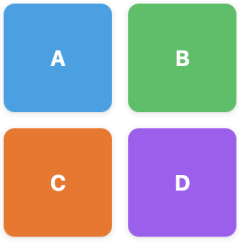
Create a `PuzzlePiece` component and render all pieces using .map().



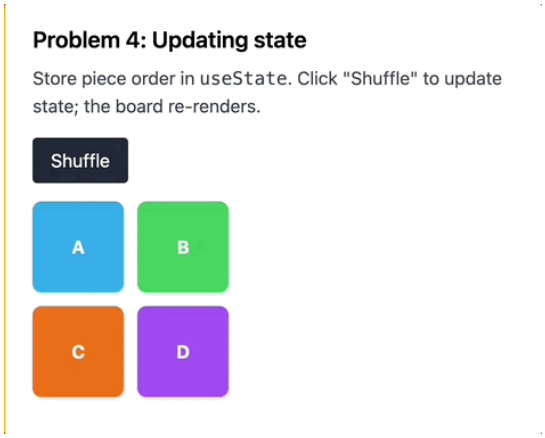
PROBLEM 3 : Board

Problem 3: Moving data between components

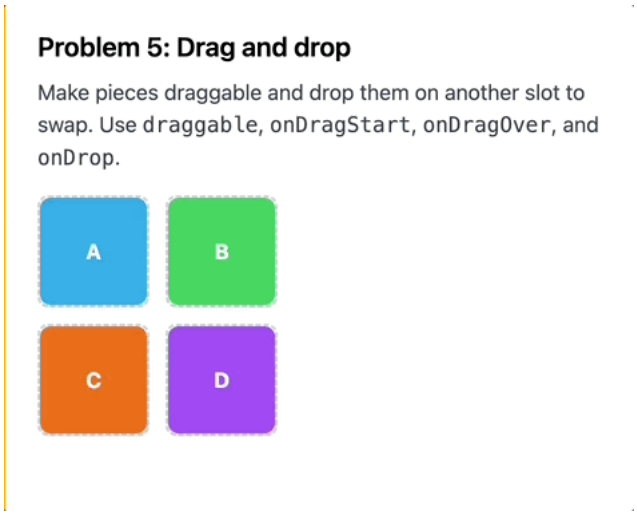
Pass `pieces` from Problem3 to `Board`; Board passes each `piece` to `PuzzlePiece`.



PROBLEM 4: SHUFFLING

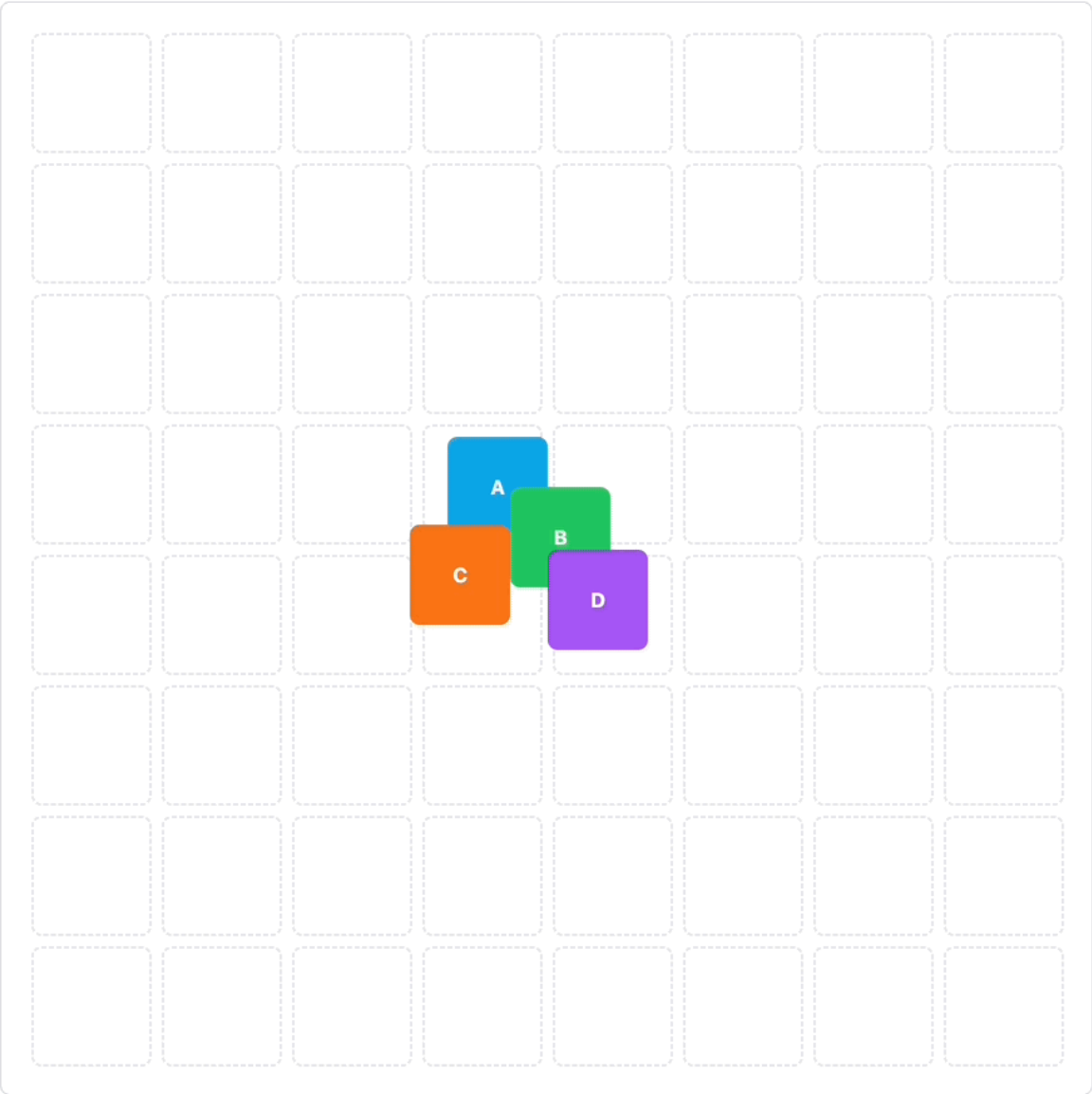


PROBLEM 5: DIRECT MANIPULATION



Problem 6: Snap to grid

The board is a large 8×8 grid. Puzzle pieces start in a jumble near the center (off the grid) and snap to the nearest grid position when dropped, depending on the magnet snap power.



Submission

- **Edit the Makefile with your NAME.**
- Run make submit.
- Download the zip.
- Upload to Canvas.
- Make a ~ 30 second video walking through Problems 1-6.
    - Just show the page; if there is an interaction (button click), show what happens when you click.
- Open permissions to the video with course staff (Dr. Torres and Nasir).